

第30回 後期総復習とBronze相当確認

今回のゴール

- 第16回～第29回で学んだ Java を振り返り、自分が「書ける」「読める」ものを整理できる。
- 短いコードを読んで、実行結果やコンパイルできるかを判断できる。
- 完成したバトルを読んで、自分で小さな変更ができる。

今回、新しい文法は出てこない。

実習時のフォルダ構成

```
ワークスペース
├─ java30_sample
│   └─ src
│       └─ battle
│           └─ (完成したバトル。第29回のサンプルと同じ 9 ファイル)
└─ java30_exercise
    └─ src
        └─ battle
            ├─ (sample と同じ 9 ファイル。TODO 付き)
            └─ Bat.java ← 第4部で自分で作る
```

- 配布ファイル: プログラム/第30回/sample/battle/、プログラム/第30回/exercise/battle/
- 解答例: プログラム/第30回/answer/battle/

今日の時間の目安

時間	内容
10 分	第1部:後期の振り返り
30 分	第2部:Bronze 相当の確認(14 問)
15 分	第3部:答え合わせと解説
25 分	第4部:完成したバトルを変更する
10 分	振り返り

- 第4部の時間を必ず残す。第2部・第3部が長引きそうな場合は、解説を間違いの多い問題にしぼる。
- 時間が足りない場合は、第4部の変更1・変更2までを必須とし、変更3(こうもり)は発展(任意)として扱ってよい。

第1部 後期の振り返り

回	できるようになったこと	主な Java
16	まず動いた	Pleiades、変数、 <code>if</code> 、 <code>while</code> 、 <code>%</code> 、 <code>break</code>
17	選べる・くり返せるようになった	<code>Scanner</code> 、配列、 <code>for</code> 、 <code>switch</code> 、 <code>do-while</code>
18	処理をメソッドへ整理できた	メソッド、引数、戻り値、キャスト、オーバーロード
19	キャラクターをクラスにまとめられた	クラス、フィールド、コンストラクタ、 <code>this</code> 、参照
20	HP を安全に扱えるようになった	<code>private</code> 、 <code>public</code> 、 <code>getter</code> 、 <code>final</code>
21	勇者と敵の違いを作れた	<code>extends</code> 、 <code>super</code> 、オーバーライド
22	種類が違っててもまとめて扱えた	ポリモフィズム、拡張 <code>for</code> 、 <code>instanceof</code>
23	共通ルールを強制できた	抽象クラス、抽象メソッド
24	人数を増減できた	<code>ArrayList</code> 、 <code>import</code>
25	クラスを整理し、API を利用できた	<code>package</code> 、 <code>Random</code> 、 <code>static</code> 、 <code>Math</code>
26	壊れたコードを自分で直せた	エラーの 3 種類、例外、 <code>try-catch</code>
27	バトルとして一通り動いた	<code>String.format</code> 、メソッドの使い回し
28	新しい能力を追加できた	<code>interface</code> 、 <code>implements</code>
29	自分で仕様を考えて変更できた	仕様 → 変更箇所 → 実装 → 確認

表を見て、次の 3 つに分けてノートに書く。

- 自分で書ける
- 読めば分かる
- まだ自信がない

第2部 Bronze 相当の確認(14問)

何も見ずに解く。答えは第3部で確認する。

型・演算子

問1 出力は何か。

```
System.out.println(10 % 4 + 10 / 4);
```

問2 出力は何か。

```
System.out.println(1 + 2 + "HP" + 1 + 2);
```

問3 コンパイルエラーになるものを 1 つ選ぶ。また、C の `z` の値を答える。

```
int x = 5.0;           // A
double y = 5;          // B
int z = (int) 5.9;     // C
```

分岐・繰り返し

問4 出力は何か。

```
int command = 2;
switch (command) {
    case 1:
        System.out.print("攻撃 ");
    case 2:
        System.out.print("防御 ");
    case 3:
        System.out.print("逃走 ");
        break;
    default:
        System.out.print("不明 ");
}
```

問5 出力は何か。

```
int n = 0;
for (int i = 1; i <= 10; i += 3) {
    n++;
}
System.out.println(n);
```

配列・メソッド

問6 次のコードのまちがいを見つけて、直す。

```
int[] hp = { 30, 20, 40 };
for (int i = 0; i <= hp.length; i++) {
    System.out.println(hp[i]);
}
```

問7 出力は何か。

```
public static void main(String[] args) {
    int level = 1;
    levelUp(level);
    System.out.println(level);
}

static void levelUp(int level) {
    level++;
}
```

クラス・カプセル化・static

問8 出力は何か。

```
class Item {
    String name;

    Item(String name) {
        this.name = name;
    }
}
```

```
Item a = new Item("薬草");
Item b = a;
b.name = "毒消し";
System.out.println(a.name);
```

問9 `Character` の `hp` は `private` である。`Main` で次のように書いた。コンパイルできるか。できないなら、HP を読むにはどう書けばよいか。

```
System.out.println(hero.hp);
```

問10 出力は何か。

```
class Counter {
    static int total = 0;
    int mine = 0;

    Counter() {
        total++;
        mine++;
    }
}
```

```
new Counter();
new Counter();
Counter x = new Counter();
System.out.println(Counter.total + " " + x.mine);
```

問11 出力は何か。

```
class Monster {
    String sound() {
        return "...";
    }

    void cry() {
        System.out.println(sound());
    }
}

class Slime extends Monster {
    @Override
    String sound() {
        return "ぷるぷる";
    }
}
```

```
Monster m = new Slime();
m.cry();
```

問12 次のうち、コンパイルできるものを 1 つ選ぶ。

- A: `abstract class Boss { }` を作り、`Boss b = new Boss();` と書く。
- B: `interface Healer { void cure(); }` を `implements` したクラスで、`void cure() { }` と書く (`public` なし)。
- C: `class Knight implements Healer, Guard { ... }` のように、2 つのインタフェースを `implements` し、両方のメソッドを `public` で書く。

package・import・ArrayList

問13 `import` を書かないと使えないものを、すべて選ぶ。

`String` `ArrayList` `Math` `Random` (同じパッケージの) `Hero`

問14 出力は何か。

```
ArrayList<String> list = new ArrayList<>();
list.add("勇者");
list.add("戦士");
list.add("僧侶");
list.remove(1);
list.add("魔法使い");
System.out.println(list.size() + " " + list.get(1));
```

第3部 答え合わせと解説

■ 答え(第2部の後で開く)

問	答え	ポイント(学んだ回)
1	4	<code>10 % 4</code> は 2、 <code>10 / 4</code> は 2(整数の割り算)(16)
2	3HP12	左から計算。 <code>1 + 2</code> は先に 3。文字列の後は、つながるだけ(16)
3	A、 <code>z</code> は 5	<code>double</code> → <code>int</code> はキャストが必要。小数点以下は切り捨て(18)
4	防御 逃走	<code>case 2</code> から、 <code>break</code> があるところまで実行される(17)
5	4	<code>i</code> は 1、4、7、10(17)
6	<code><=</code> を <code><</code> にする	最後の添字は <code>length - 1</code> 。このままだと実行時エラー(17、26)
7	1	引数は値のコピー。戻り値で返さないと元の変数は変わらない(18)
8	毒消し	<code>a</code> と <code>b</code> は同じインスタンスを指す(19)
9	できない。 <code>hero.getHp()</code> と書く	<code>private</code> はクラスの外から使えない。 <code>getter</code> を使う(20)
10	3 1	<code>static</code> 変数は全員で 1 つ。 <code>mine</code> はインスタンスごと(25)
11	ぶるぶる	<code>cry</code> は <code>Monster</code> のメソッドだが、中で呼ぶ <code>sound</code> は中身の <code>Slime</code> のもの(21、22)
12	C	A:抽象クラスは <code>new</code> できない(23)。B: <code>public</code> が必要(28)。C:インタフェースは複数 <code>implements</code> できる(28)
13	<code>ArrayList</code> 、 <code>Random</code>	<code>String・Math</code> は <code>java.lang</code> 。同じパッケージのクラスも <code>import</code> 不要(24、25)
14	3 僧侶	1 番の「戦士」を消すと「僧侶」が 1 番につめられる。最後に「魔法使い」が加わる(24)

- 間違えた問題の「学んだ回」のレジメを読み直す。
- コードを Pleiades に入力して実行し、答えを自分で確かめてもよい(問3・6・9・12 は、エラーになることを確かめる)。

第4部 完成したバトルを変更する

`java30_exercise` を作り、`battle` パッケージに `プログラム/第30回/exercise/battle/` の 9 ファイルを入れる。まず `java30_sample` と同じように動くことを確認してから、`// TODO` の 3 つの変更を行う。

(完成したバトル。乱数のため、実行結果は毎回変わる)

変更1 ピンチの勇者は強くなる(Lv1)

- `Hero` の `attack` で、「3 ターンごと」に加えて、「HP が最大 HP の半分以下のとき」も会心の一撃にする。

ヒント: `||` (または) を使う (第17回)。

変更2 たおした敵の数(Lv2)

- たおした敵の数を数える変数を用意し、敵をたおすたびに 1 増やす。
- 勝ったときに、`たおした敵: 5体` のように表示する。

ヒント: 経験値を足しているところ (`totalExp += ...`) の近くで数える。

変更3 2 回攻撃するこうもり(Lv3)

- 新しいクラス `Bat` を作る。 `Enemy` を受けつぎ、`attack` で `normalAttack(target)` を 2 回呼ぶ。
- `Main` の敵のグループに、こうもり (HP 15、攻撃力 3、経験値 8) を追加する。

完成条件:

敵: 5体

- 1: スライム HP: 20/20
- 2: ゴブリン HP: 40/40
- 3: オーク HP: 70/70
- 4: いやしスライム HP: 25/25
- 5: こうもり HP: 15/15

- こうもりの攻撃では、`こうもりの攻撃!` が 2 回続けて表示される。
- 勝ったとき、たおした敵の数が表示される。

発展(任意)

- 第29回の機能カードから、まだやっていないものを 1 つ追加する。
- 今日の変更で、バトルが強くなりすぎたり弱くなりすぎたりしていないか、何回か遊んで数値を調整する。

振り返り

- 第1部の表で、「自分で書ける」に入れたものはいくつあったか。
- 第2部で間違えた問題は、どの回の内容だったか。
- 後期のはじめ(第16回)のプログラムと、今日のプログラムを比べて、何ができるようになったか。
- これから Java で作ってみたいものは何か。